

Assignment Number: 2 (a & b)**Name:** Atharv Santosh Danave**Roll No.:** 11**Assignment Title:** Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms.**CODE**

```
// =====  
// Measures and compares sequential vs parallel performance  
// =====  
  
#include <iostream>  
#include <vector>  
#include <algorithm>  
#include <omp.h>  
#include <chrono>  
#include <random>  
using namespace std;  
  
// -----  
// BUBBLE SORT  
// -----  
  
void bubbleSort_Sequential(vector<int>& arr) {  
    int n = arr.size();  
    for (int i = 0; i < n - 1; i++)  
        for (int j = 0; j < n - i - 1; j++)  
            if (arr[j] > arr[j+1])  
                swap(arr[j], arr[j+1]);  
}  
  
// Odd-Even Transposition Sort (parallelisable bubble sort variant)  
void bubbleSort_Parallel(vector<int>& arr) {  
    int n = arr.size();  
    for (int phase = 0; phase < n; phase++) {  
        if (phase % 2 == 0) {  
            // Even phase: compare (0,1), (2,3), (4,5) ...  
            #pragma omp parallel for schedule(static)  
            for (int i = 0; i < n - 1; i += 2) {  
                if (arr[i] > arr[i+1])  
                    swap(arr[i], arr[i+1]);  
            }  
        }  
    }  
}
```

```

    }
} else {
    // Odd phase: compare (1,2), (3,4), (5,6) ...
    #pragma omp parallel for schedule(static)
    for (int i = 1; i < n - 1; i += 2) {
        if (arr[i] > arr[i+1])
            swap(arr[i], arr[i+1]);
    }
}
}
}

```

```

// MERGE SORT
//

```

```

void merge(vector<int>& arr, int l, int m, int r) {
    vector<int> left(arr.begin()+l, arr.begin()+m+1);
    vector<int> right(arr.begin()+m+1, arr.begin()+r+1);
    int i = 0, j = 0, k = l;
    while (i < (int)left.size() && j < (int)right.size())
        arr[k++] = (left[i] <= right[j]) ? left[i++] : right[j++];
    while (i < (int)left.size()) arr[k++] = left[i++];
    while (j < (int)right.size()) arr[k++] = right[j++];
}

```

```

void mergeSort_Sequential(vector<int>& arr, int l, int r) {
    if (l >= r) return;
    int m = (l + r) / 2;
    mergeSort_Sequential(arr, l, m);
    mergeSort_Sequential(arr, m+1, r);
    merge(arr, l, m, r);
}

```

```

void mergeSort_Parallel(vector<int>& arr, int l, int r, int depth = 0) {
    if (l >= r) return;
    int m = (l + r) / 2;

```

```

if (depth < 4) { // spawn threads up to depth 4
    #pragma omp parallel sections
    {
        #pragma omp section
        mergeSort_Parallel(arr, l, m, depth+1);
        #pragma omp section
        mergeSort_Parallel(arr, m+1, r, depth+1);
    }
} else {
    mergeSort_Sequential(arr, l, m);
    mergeSort_Sequential(arr, m+1, r);
}
merge(arr, l, m, r);
}

```

```

// UTILITY
//

```

```

vector<int> generateRandom(int n) {
    vector<int> v(n);
    mt19937 rng(42);
    uniform_int_distribution<int> dist(1, 100000);
    for (auto& x : v) x = dist(rng);
    return v;
}

```

```

bool isSorted(const vector<int>& v) {
    for (int i = 0; i + 1 < (int)v.size(); i++)
        if (v[i] > v[i+1]) return false;
    return true;
}

```

```

void printArr(const vector<int>& v, int limit = 10) {
    for (int i = 0; i < min((int)v.size(), limit); i++)
        cout << v[i] << " ";
    if ((int)v.size() > limit) cout << "...";
    cout << endl;
}

```

```

}

// -----
// MAIN
// -----

int main() {
    const int BUBBLE_N = 5000;
    const int MERGE_N = 500000;

    cout << "===== " << endl;
    cout << " Parallel Sorting Algorithms using OpenMP " << endl;
    cout << "===== " << endl;

```

```

// — Bubble Sort —

```

```

cout << "\n--- Bubble Sort (n = " << BUBBLE_N << ") ---" << endl;

```

```

auto orig = generateRandom(BUBBLE_N);

```

```

auto arr1 = orig;

```

```

auto s1 = chrono::high_resolution_clock::now();

```

```

bubbleSort_Sequential(arr1);

```

```

auto e1 = chrono::high_resolution_clock::now();

```

```

double t_bubble_seq = chrono::duration<double, milli>(e1-s1).count();

```

```

cout << "Sequential sorted: " << (isSorted(arr1) ? "YES" : "NO")

```

```

    << " | Time: " << t_bubble_seq << " ms" << endl;

```

```

auto arr2 = orig;

```

```

auto s2 = chrono::high_resolution_clock::now();

```

```

bubbleSort_Parallel(arr2);

```

```

auto e2 = chrono::high_resolution_clock::now();

```

```

double t_bubble_par = chrono::duration<double, milli>(e2-s2).count();

```

```

cout << "Parallel sorted: " << (isSorted(arr2) ? "YES" : "NO")

```

```

    << " | Time: " << t_bubble_par << " ms" << endl;

```

```

cout << "Speedup (Bubble): " << t_bubble_seq / t_bubble_par << "x" << endl;

```

```

// — Merge Sort —

```

```

cout << "\n--- Merge Sort (n = " << MERGE_N << ") ---" << endl;

auto big = generateRandom(MERGE_N);

auto arr3 = big;
s1 = chrono::high_resolution_clock::now();
mergeSort_Sequential(arr3, 0, (int)arr3.size()-1);
e1 = chrono::high_resolution_clock::now();
double t_merge_seq = chrono::duration<double, milli>(e1-s1).count();
cout << "Sequential sorted: " << (isSorted(arr3) ? "YES" : "NO")
    << " | Time: " << t_merge_seq << " ms" << endl;

auto arr4 = big;
s2 = chrono::high_resolution_clock::now();
mergeSort_Parallel(arr4, 0, (int)arr4.size()-1);
e2 = chrono::high_resolution_clock::now();
double t_merge_par = chrono::duration<double, milli>(e2-s2).count();
cout << "Parallel sorted: " << (isSorted(arr4) ? "YES" : "NO")
    << " | Time: " << t_merge_par << " ms" << endl;
cout << "Speedup (Merge): " << t_merge_seq / t_merge_par << "x" << endl;

cout << "\nFirst 10 elements of sorted array: ";
printArr(arr4);

return 0;
}

// — Compile & Run —————
// g++ -fopenmp -O2 -o a2 assignment2_sorting.cpp && ./a2

```

OUTPUT

```
(no subject) - practicalcodes04@gmail.com - Gmail — Mozilla Firefox
atharv1104@kali: ~/Downloads

$ g++ -fopenmp -O2 -o a2 assignment2_sorting.cpp 66 ./a2
/usr/bin/ld: error in /usr/lib/gcc/x86_64-linux-gnu/14/../../../../x86_64-linux-gnu/Scrt1.o(.sframe); no .sframe will be created

Parallel Sorting Algorithms using OpenMP

— Bubble Sort (n = 5000) —
Sequential sorted: YES | Time: 47.6795 ms
Parallel sorted: YES | Time: 63.8385 ms
Speedup (Bubble): 0.746877x

— Merge Sort (n = 500000) —
Sequential sorted: YES | Time: 75.3124 ms
Parallel sorted: YES | Time: 39.297 ms
Speedup (Merge): 1.91649x

First 10 elements of sorted array: 1 1 1 1 1 1 1 1 2 2 ...

(atharv1104@kali)~-[~/Downloads]
```